

This is a phenomenal way to introduce your students to **API Security**! In the real world, open endpoints are vulnerabilities; websites require an API Key to trace who is making requests, prevent abuse, and manage rate limiting.

By shifting from hardcoded keys to checking a **MongoDB collection**, your students will learn how dynamic authentication works in modern production systems.

Here is a complete, professional, ready-to-print **Lab Sheet** for this exercise.

CS-3020: Web Application Development

Lab 05: Securing REST APIs using MongoDB-Backed API Keys

Learning Objectives

By the end of this practical session, you should be able to:

1. **Design an Authentication Document Schema** in MongoDB to securely store and manage valid API Keys.
2. **Implement Interceptor/Filter Patterns** (via Spring Component Architecture) to read HTTP Request Headers.
3. **Cross-Reference Requests dynamically** against a database collection before allowing access to business logic.

Prerequisites

- A completed, working version of **Lab 04** (Temperature Converter with functioning Service, Repository, and Controller layers).
- access to MongoDB Compass, Mongo Shell, or your application's setup to insert a test collection.

Exercise 1: Preparing the API Key Collection

Task Description

Before your application can validate keys, it needs a dedicated collection in MongoDB to store valid consumers. You will create an ApiKey domain model, its matching repository, and insert sample access tokens into your database.

Step 1: Create the API Key Model (ApiKey.java)

Create a new file in your model package named ApiKey.java. This maps directly to a collection named api_keys.

```
package com.example.tempconverter.model;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;

@Document(collection = "api_keys")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class ApiKey {
    @Id
    private String id;
    private String keyValue; // The actual secret token string
    private String clientName; // Descriptive identifier (e.g., "MobileAppClient")
    private boolean active; // Quickly revoke access without deleting the
    document
}
```

Step 2: Create the API Key Repository (ApiKeyRepository.java)

Create an interface inside your repository package. You will need a method to look up a key by its exact string value.

```
package com.example.tempconverter.repository;

import com.example.tempconverter.model.ApiKey;
import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;
import java.util.Optional;

@Repository
public interface ApiKeyRepository extends MongoRepository<ApiKey, String> {
    // Spring Boot auto-generates a query to find the key and verify it is active
    Optional<ApiKey> findByKeyValueAndActiveTrue(String keyValue);
}
```

Step 3: Populating Test Keys in MongoDB

You must manually seed your database with at least two active keys.

If using **MongoDB Compass** or a shell, execute an insert inside your database to add active records:

```
[
  {
    "keyValue": "SUPER-SECRET-DEV-KEY-123",
    "clientName": "Frontend-Web-App",
    "active": true
  },
  {
    "keyValue": "EXPIRED-HACKER-KEY-999",
    "clientName": "Suspicious-Client",
    "active": false
  }
]
```

Exercise 2: Implementing the Security Validation Layer

Task Description

Instead of allowing requests directly to the business controllers, you will intercept incoming web traffic, inspect the HTTP headers for a parameter called X-API-KEY, cross-reference it with MongoDB, and block unauthorized traffic.

Step 1: Creating a Custom Security Exception

To return standard errors when validation fails, define a clean exception class in a new package called exception.

```
package com.example.tempconverter.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.UNAUTHORIZED) // Automatically returns HTTP 401
Unauthorized status
public class UnauthorizedException extends RuntimeException {
    public UnauthorizedException(String message) {
        super(message);
    }
}
```

Step 2: Updating the Service Layer for Key Check

Open your existing TemperatureService.java class and modify it to inject your new repository and add a validation utility method.

```
import com.example.tempconverter.repository.ApiKeyRepository;
import com.example.tempconverter.exception.UnauthorizedException;

// 2. Modify TemperatureService to include ApiKeyRepository
@Service
@RequiredArgsConstructor
public class TemperatureService {

    private final com.example.tempconverter.repository.TemperatureRepository
temperatureRepository;
    private final ApiKeyRepository apiKeyRepository; // INJECT THE NEW REPOSITORY

    // 3. Add this validation method
    public void validateApiKey(String requestKey) {
        if (requestKey == null || requestKey.trim().isEmpty()) {
            throw new UnauthorizedException("API Key missing from HTTP Headers!");
        }

        // Search the database for an active matching key token
        apiKeyRepository.findByKeyValueAndActiveTrue(requestKey.trim())
            .orElseThrow(() -> new UnauthorizedException("Invalid, inactive, or
revoked API Key provided!"));
    }

    // ... rest of your existing conversion logic remains untouched ...
}
```

Step 3: Safeguarding Controller Endpoints

Open your `TemperatureController.java` class. You will now use `@RequestHeader` to demand an API Key header value on incoming requests before executing conversion logic.

```
package com.example.tempconverter.controller;

import com.example.tempconverter.model.TemperatureLog;
import com.example.tempconverter.service.TemperatureService;
import org.springframework.web.bind.annotation.*;
import lombok.RequiredArgsConstructor;
import java.util.List;

@RestController
@RequestMapping("/api/temperatures")
@RequiredArgsConstructor
public class TemperatureController {

    private final TemperatureService temperatureService;

    @PostMapping("/convert")
    public TemperatureLog convertTemperature(
        @RequestHeader("X-API-KEY") String apiKey, // Captures key from HTTP
        @RequestParam double value,
        @RequestParam String unit
    ) {
        // Enforce the database cross-reference check
        temperatureService.validateApiKey(apiKey);

        return temperatureService.convertAndSave(value, unit);
    }

    @GetMapping("/history")
    public List<TemperatureLog> getAllLogs(
        @RequestHeader("X-API-KEY") String apiKey // Secures history viewing as
        well
    ) {
        temperatureService.validateApiKey(apiKey);
        return temperatureService.getAllLogs();
    }
}
```

Exercise 3: Verification & Testing

Because web browsers cannot easily customize HTTP header blocks directly out of the box, you must test this using **Postman**, **Insomnia**, or a console command like `curl`.

Test Case 1: Missing API Key Header

- **Action:** Send a POST request to `http://localhost:8080/api/temperatures/convert?value=25&unit=Celsius` with **no headers**.
- **Expected Result:** HTTP Status 401 Unauthorized or 400 Bad Request specifying that header X-API-KEY is missing.

Test Case 2: Expired/Invalid API Key

- **Action:** Send a request adding a custom header row named X-API-KEY with the value EXPIRED-HACKER-KEY-999.
- **Expected Result:** HTTP Status 401 Unauthorized. (Database check failed because active field is false).

Test Case 3: Valid Authorized Request

- **Action:** Set the X-API-KEY header value to SUPER-SECRET-DEV-KEY-123.
- **Expected Result:** HTTP Status 200 OK with your expected conversion JSON document returned!